

signal_detector

Classroom phone-presence detector — setup and operating manual

July 24, 2026

1 What it is

Two ESP32-S3 boards act as passive 2.4 GHz radio sensors. Each one listens for WiFi and Bluetooth activity, and every ~ 2 s reports what it heard to a Python program (`collector.py`) running in Termux on your phone. The phone keeps a *baseline* of the devices normally present in the empty room, then shows a live table of any *extra* devices, using the two boards to place each one in a rough corner of the room. It also tracks how much each device transmits and flags short *episodes* of use — a phone picked up for a moment — buzzing you when one starts near a board, and leaving a “used Xs ago” marker so you can step out for a few minutes and review what happened when you return. A web page (served by the phone) shows the same list, lets you mark a device *watch* or *disregard*, and graphs any device’s traffic over several time windows (5 min up to 1 h 40 min), with episodes shaded.

Read this before trusting it. A phone that is *off* or in *airplane mode* emits nothing and is invisible — which is exactly what you tell students to do, so a careful cheater defeats it for free. Modern phones rotate randomized MAC addresses, so the device *count* is inflated (one phone can appear as several rows). Location is corner-level, never a single desk. Treat this as a live map of *active radios* plus a deterrent — not as proof, and not as a substitute for collecting phones at the door.

2 What you need

- **2× ESP32-S3-DevKitC-1** boards (WROOM-1).
- 2 USB-C cables and 2 power sources (power banks or chargers) for the room.
- A laptop with the **Arduino IDE** (only to flash the boards, once).
- An **Android phone** with **Termux** and **Termux:API** (both from F-Droid — see the note below).
- A WiFi network the phone and both boards can all join.

Install **Termux** and **Termux:API** from the *same* source (both from F-Droid). They must be signed by the same key or the notifications will silently not work. Do not mix F-Droid and Play Store builds.

3 Part A — Flash the two sensors (once)

1. In Arduino IDE: install **esp32** by **Espressif** (Boards Manager) and **NimBLE-Arduino** (Library Manager, version 2.x).
2. Select board **ESP32S3 Dev Module**. Set **USB CDC On Boot: Enabled** (so Serial works over USB-C).
3. Edit `secrets.h` with your WiFi and your phone’s IP address (you get that IP in Part B):

```
#define WIFI_SSID    "your-wifi"
#define WIFI_PASS    "your-password"
#define BOARD_ID     1           // 1 on the first board, 2 on the second
#define SERVER_HOST  "192.168.1.50" // your phone’s LAN IP
#define SERVER_PORT  8000
```

- Flash the first board with `BOARD_ID 1`. Then change it to `BOARD_ID 2` and flash the second board.
- Label the boards “1” and “2”. Board 1 goes front-left, board 2 goes back-right (or edit `BOARD_LABELS` in `collector.py` to match where you actually place them).

The boards have no buttons and no commands — once powered, they just sniff and POST. All control is on the phone.

4 Part B — Set up the phone (Termux)

- Install the packages and the Python web framework:

```
pkg install python termux-api
pip install flask
```

- Find the phone’s LAN IP (the `inet` on `wlan0`) and put it in `secrets.h` as `SERVER_HOST` before flashing the boards:

```
ifconfig
```

- Copy `collector.py` onto the phone (git clone the repo, or paste the file).
- Read the built-in help any time:

```
python collector.py --help
```

5 Part C — Running an exam

- Power both boards, one in each corner.
- On the phone, keep it plugged in, and in Termux:

```
termux-wake-lock      # keeps it running with the screen off
python collector.py
```

- Watch the table start to populate (that confirms the boards are reaching the phone).
- With the room still **empty**, press `[b]` then Enter. It captures the ambient devices for 10 minutes, then switches to live automatically.
- Let the students in. The table now lists only the *extra* devices, with a corner and a type. When a device is actively *used* (a traffic episode) near a board, the phone buzzes. Glance at the zone and walk that way; the `USED` column and the graph keep the moment if you look later.

If Android keeps killing Termux during a long exam: keep the phone plugged in, run `termux-wake-lock`, and set Termux to *Unrestricted* under Android’s battery settings.

6 Reading the live table

Column	Meaning
MARK	Blank, W (watch), or D (disregard). Set on the web page.
MAC	Device hardware address (often randomized, so it changes over time).
RSSI	Strongest signal seen, in dBm. Closer to 0 = nearer.
ZONE	near front-left / near back-right / center .
TRAFFIC	Frames and bytes heard from this MAC in the last ~2s (WiFi only). A relative, undersampled “h
USED	NOW during a traffic episode, else “used Xs ago” for a few minutes after one; present for a BLE de
TYPE	WiFi or BLE, plus rand (randomized MAC) or apple .

A device is placed in a corner only when *both* boards hear it: the board with the stronger signal wins; if they are within a few dB it is called **center**. Watched (W) devices sort to the top and show red; disregarded (D) devices sort to the bottom and show green.

7 Episodes — catching a brief moment of use

The point of the exam isn’t a phone being *on*, it’s a phone being *used*: picked up, a query sent, an answer read, put down — a burst of tens of seconds (a query, or a photo of the question uploaded). The collector detects this as an **episode**: over the last 30s, a device’s WiFi traffic must clear two gates at once —

- **volume**: total bytes above a floor (EPISODE_MIN_BYTES), and
- **average frame size** above EPISODE_MIN_FRAME.

The frame-size gate is what separates real use from idle. An idle-but-connected phone still emits a trickle of *tiny* keepalive/ACK frames; a real transfer sends *large* data frames. Requiring a large average frame size means a device merely holding its WiFi connection alive never trips, while a genuine query does. 30s matches the length of a brief use.

Each episode is **logged with a timestamp**, so this survives you leaving the room: step out to the bathroom for two or three minutes, and when you come back the device still shows “used 90s ago” and its graph shows a shaded hump at that time. When an episode starts on a device close to a board (or one you marked W), the phone buzzes.

BLE presence. Watches and earbuds show up over Bluetooth and don’t generate WiFi traffic — for them the signal is simpler: a non-baseline BLE device is a violation *just by being present*. Those rows show yellow, are labelled **present**, sort up, and buzz you. (Since students shouldn’t have any such device, its mere presence is the alert.)

Calibrating the floor

Do this once, in the real room, with a test phone that is *not* in the baseline. Select it on the web page and read the **calib** line under the graph — it shows the current 30s window sum and average frame size, next to the floor and frame thresholds, and says **EPISODE** or **below**.

1. **Idle**: leave the phone connected but unused for a few minutes. Note the highest window sum it reaches (keepalive chatter).
2. **Query**: do a real query (and a photo upload) a few times; note the window sum.
3. Set EPISODE_MIN_BYTES **between** the idle peak and the smallest query. The frame-size gate handles the rest.

Calibrate from a worst-case desk — far from both boards. A phone far from a sensor is heard weaker and its frames are caught less often, so the *same* query yields a smaller window sum than one done next to a board. Set the floor from the far position or a corner query can slip under it.

8 Web dashboard and traffic graphs

The collector serves a small web page. On the phone open `http://localhost:8000/`; from your laptop on the same WiFi open `http://<phone-ip>:8000/`. Nothing is installed and no internet is needed. It gives you three things the console can't:

- **Marking.** Each row has W and D buttons. Mark a device W (watch) and it turns red and sorts to the top; mark it D (disregard — e.g. your own phone, or one you already checked) and it turns green and sorts to the bottom. Unmarked devices sit in the middle. Marks persist across restarts (`marks.json`).
- **Traffic graphs with time windows.** Click a MAC to graph its traffic. Buttons pick the window: 5m, 15m, 30m, 60m, 1h40m. The short windows are for reading a single episode; the long one is the whole picture.
- **Episodes shaded.** Detected episodes appear as shaded humps on the graph, so you can see exactly when a device was active.
- **Calib readout.** Below the graph, a `calib` line shows the selected device's current 30 s window sum and average frame size against the thresholds — used to calibrate the floor (see above).

The long graph is only meaningful for *stable* MACs (no `rand` tag) — a laptop, or a phone associated to an access point. A randomized MAC is discarded and replaced by the phone every few minutes, so its history only ever spans those few minutes. You are watching a short-lived identity, not a device across the whole exam.

9 collector.py — command reference

Type the letter in the Termux console, then press Enter.

Key	Action
<code>b</code>	Start baseline capture (10 min). Run in the empty room.
<code>l</code>	Live detection.
<code>i</code>	Idle (keep receiving, stop deciding).
<code>s</code>	Save baseline to <code>baseline.json</code> .
<code>c</code>	Clear baseline.
<code>+</code>	Raise RSSI threshold by 5 dBm (fewer, closer devices).
<code>-</code>	Lower RSSI threshold by 5 dBm (more, farther devices).
<code>q</code>	Quit.

Tunables (top of `collector.py`)

Constant	Meaning
<code>RSSI_THRESHOLD</code>	Ignore devices weaker than this (default -80 dBm).
<code>ALERT_STRONGEST_MIN</code>	Only buzz on episodes at least this close (-70 dBm), unless W.
<code>ALERT_COOLDOWN_SECS</code>	Minimum gap between buzzes, per MAC (default 60).
<code>BASELINE_SECS</code>	Baseline capture length (default 600 = 10 min).
<code>EXPIRE_SECS</code>	Forget a device unseen this long (default 120).
<code>BOARD_LABELS</code>	Which corner each board sits in.
<code>HISTORY_SECS</code>	How much traffic history is kept (default 6000 = 1 h 40 min).
<code>EPISODE_WINDOW_SECS</code>	Detection window (default 30 s; a brief phone use).
<code>EPISODE_MIN_BYTES</code>	Window traffic must clear this floor (bytes/30 s). Calibrate.
<code>EPISODE_MIN_FRAME</code>	... and average frame size must clear this (default 250 B).
<code>EPISODE_RETAIN_SECS</code>	Keep “used Xs ago” this long after an episode (default 300).

10 Troubleshooting

Symptom	Fix
Table stays empty	Boards can't reach the phone. Check <code>SERVER_HOST</code> matches the phone's current IP, both boards and the phone are on the same WiFi, and the phone isn't on mobile data. On the board's Serial you should see <code>[POST] ... HTTP 200</code> .
No notification buzz	Install the Termux:API app (not just the <code>termux-api</code> package) from F-Droid, same source as Termux.
Termux dies mid-exam	Plug in the phone, run <code>termux-wake-lock</code> , set Termux battery to Unrestricted.
Huge device count	Randomized MACs. Raise the threshold with <code>[+]</code> , and read the count as “activity”, not “number of phones”.
Everything shows in one corner	The boards may be too close together or one is dead. Separate them; check both show <code>HTTP 200</code> .

11 Honest limits

- **Off / airplane mode = invisible.** No radio, no detection.
- **Randomized MACs inflate the count.** One phone → several rows.
- **Zoning is coarse.** A corner, not a desk; bodies and reflections blur the signal.
- **2.4 GHz only.** A phone using 5 GHz WiFi only is not seen.

It is a proctoring aid and a deterrent. The only method that actually removes phones is collecting them physically at the door.